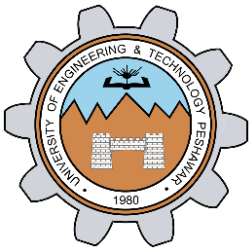
Lab # 04

Quick sort and Merge sort



Spring 2026

Data Structures and Algorithms Lab

Submitted by: **M Mohsin Khan**

Registration No.: **24PWCSE2382**

Class Section: **C**

Submitted to:

Engr. Abdullah Hamid

June 19rd , 2026

Department of Computer Systems Engineering

University of Engineering and Technology, Peshawar

Task 01: Implementation of Quick Srot:

```cpp
#include <iostream>
#include <vector>
using namespace std;

// Partition function (Lomuto partition scheme)
int partition(vector<int> &array, int start, int end)
{
    int pivot = array[end]; // Last element as pivot
    int index = start - 1;  // Smaller element boundary

    for (int current = start; current < end; current++)
    {
        if (array[current] <= pivot)
        {
            index++;
            swap(array[current], array[index]);
        }
    }

    // Place pivot in correct position
    index++;
    swap(array[end], array[index]);

    return index; // Pivot index
}

// Quick Sort function
void quickSort(vector<int> &array, int start, int end)
{
    if (start < end)
    {
        int pivotIndex = partition(array, start, end);

        // Left subarray
        quickSort(array, start, pivotIndex - 1);

        // Right subarray
        quickSort(array, pivotIndex + 1, end);
    }
}

// Print function
void printArray(const vector<int> &array)
{
    for (int value : array)
    {
        cout << value << " ";
    }
    cout << endl;
}

// Main function
int main()
{
    vector<int> array = {12, 31, 35, 8, 32, 17};

    cout << "Original array: ";
    printArray(array);

    quickSort(array, 0, array.size() - 1);

    cout << "Sorted array: ";
    printArray(array);

    return 0;
}
```

Terminal Output:

```
PS D:\4th Semester\DSA\lab_04_mergeSort_quickSort> g++ .\quick_sort.cpp
PS D:\4th Semester\DSA\lab_04_mergeSort_quickSort> .\a.exe
Original array: 12 31 35 8 32 17
Sorted array: 8 12 17 31 32 35
```

Task 02: Merge Sort implementation:

```cpp
#include <iostream>
using namespace std;
void merge(int arr[], int left, int mid,
int right)
{
    int n1 = mid - left + 1;
    int n2 = right - mid;

    // Create temporary arrays
    int *leftArr = new int[n1];
    int *rightArr = new int[n2];

    // Copy data to temporary arrays
    for (int i = 0; i < n1; i++)
        leftArr[i] = arr[left + i];

    for (int j = 0; j < n2; j++)
        rightArr[j] = arr[mid + 1 + j];

    int i = 0;    // Initial index of first
subarray
    int j = 0;    // Initial index of
second subarray
    int k = left;
    while (i < n1 && j < n2)
    {
        if (leftArr[i] <= rightArr[j])
        {
            arr[k] = leftArr[i];
            i++;
        }
        else
        {
            arr[k] = rightArr[j];
            j++;
        }
        k++;
    }
    while (i < n1)
    {
        arr[k] = leftArr[i];
        i++;
        k++;
    }
    while (j < n2)
    {
        arr[k] = rightArr[j];
        j++;
        k++;
    }

    // Free memory
    delete[] leftArr;
    delete[] rightArr;
}

// Recursive Merge Sort function
void mergeSort(int arr[], int left, int
right)
{
    if (left < right)
    {
        // Avoids overflow for very large
values
        int mid = left + (right - left) /
2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        // Combine
        merge(arr, left, mid, right);
    }
}

int main()
{
    int arr[] = {38, 27, 43, 3, 9, 82, 10};
    int n = sizeof(arr) / sizeof(arr[0]);

    mergeSort(arr, 0, n - 1);

    cout << "Sorted array: ";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";

    cout << endl;

    return 0;
}
```

```
PS D:\4th Semester\DSA\lab_04_mergeSort_quickSort> g++ .\merge_sort.cpp
PS D:\4th Semester\DSA\lab_04_mergeSort_quickSort> .\a.exe
Sorted array: 3 9 10 27 38 43 82
```

Comparison :

Case	Complexity
Best	O(n log n)
Average	O(n log n)
Worst	O(n log n)

RUBRICS (LABS/ PROJECTS):

Criteria & Point Assigned	**Outstanding 4**	**Acceptable 3**	**Considerable 2**	**Below Expectations 1**	**Score**
Valuing related to work, including responsibility, and professionalism.	Demonstrates **excellent** responsibility and professionalism in task managment. Shows **strong commitment to** quality work, meeting the deadline, and comprehensive, well-organized **effective documentation** following all guidelines with exemplary attention to detail.	Demonstrates **some** responsibility and professionalism. Displays **moderate commitment to** quality work, meets the deadline and shows **sufficient thoroughness** in documentation, adhering to guidelines, with some room for enhancement.	Demonstrates **limited effort** in responsibility and professionalism. Misses the deadline with **notable inconsistency** in quality work and shows **insufficient** thoroughness or clarity in documentation, necessitating improvements.	Demonstrates a **lack** of responsibility and professionalism. Misses the deadline and **lacks** quality work. Documentation is incomplete, disorganized, or unclear.	

Ability to apply techniques, methods, and procedures effectively.	Executes lab tasks **with exceptional skill** to design/develop solution and **shows proficiency** in lab techniques and procedures.	Executes lab tasks **with adequate skill** to design/develop solution and **shows proficiency** in lab techniques and procedures.	Executes lab tasks **with minimal skill** to design/develop solution and **shows modeate proficiency** in lab techniques and procedures.	Executes lab tasks **with limited skill** to design/develop solution and **lacks proficiency** in lab techniques and procedures.	
Demonstrate understanding and relevant Knowledge.	Demonstrates **exceptional** understanding of concepts with **clear explanation** of investigation and analysis of the objectives and outcomes.	Demonstrates a **good** understanding of concepts with **some explanations** of investigation and analysis that align with the lab's objectives and outcomes.	Demonstrates a **basic** understanding of key concepts but **lacks clarity or depth in explaining** the investigation and analysis, with some inconsistencies.	Demonstrates **limited** understanding of concepts with **unclear explanations** of investigation and analysis that do not align with the lab's objectives and outcomes.	